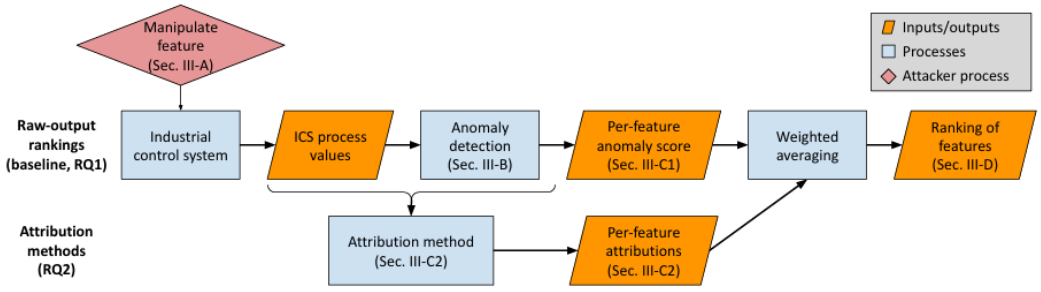
Project report: Intrinsic Explainability for ICS Anomaly Detection (by Kartik Mahnot and Udipt Saini)

ICS and Motivation

Industrial Control Systems (ICS) are used in environments such as water treatment plants, chemical industries, manufacturing systems, and power grids. These systems continuously monitor physical processes using sensors and actuators.

Machine learning models are commonly used to detect cyber attacks and abnormal behavior in such systems. However, anomaly detection alone is not enough. Operators also need to know which sensor or actuator caused the anomaly.

This is the core motivation of the paper. The paper evaluates explainability methods on ICS anomaly detection systems to determine how accurately they identify attacked sensors.



TEP Dataset

The Tennessee Eastman Process (TEP) dataset is a benchmark industrial dataset that simulates a realistic chemical process plant.

It contains:

- Multiple sensors
- Multiple actuators
- Temporal process behavior
- Predefined cyber attacks

The attacks include sensor spoofing, sensor freezing, actuator manipulation, and control signal modification. Each attack specifies the attacked sensor or actuator and attack timing. This information is used as ground truth during evaluation.

The data is multivariate time-series data, meaning the order of time is important.

CNN-Based Anomaly Detection

The baseline model is a Conv1D-based forecasting model.

The model takes:

- Previous 50 timesteps
- All sensor and actuator values

and predicts the next timestep.

Input shape:

(history, nI)

where:

- history = 50
- nI = number of sensors/features.

The model uses:

- 64 Conv1D filters
- Kernel size = 3.

The 1D convolution slides across time and learns short temporal patterns such as:

- Pressure spikes
- Valve-flow relationships
- Oscillations
- Abnormal transitions.

Architecture:

Input
↓
Conv1D
↓
Batch Normalization
↓
Conv1D
↓
Batch Normalization
↓
Flatten
↓
Dense Layer
↓
Forecast Output

The model predicts future sensor values and compares them with actual values using Mean Squared Error (MSE).

Large prediction error indicates abnormal behavior.

The original paper does not report exact reconstruction losses for the CNN model, but mentions that the benign validation reconstruction error remained below 0.25 after early stopping on the TEP dataset. In our reproduction, the CNN model achieved a final validation loss of 0.2956 after 10 epochs. The small difference may be due to variations in training epochs, initialization, preprocessing, and TensorFlow version compatibility. Despite this, the reproduced model successfully replicated the anomaly detection and explainability evaluation pipeline presented in the paper.

Explainability

The CNN detector only identifies that an anomaly occurred. It does not identify which sensor caused the anomaly.

Therefore, post-hoc explainability methods are applied.

Saliency Maps (SM)

Saliency Maps are gradient-based explainability methods.

They compute gradients of the anomaly error with respect to the input features.

This measures how sensitive the anomaly score is to each input feature.

If small changes in a sensor strongly affect the anomaly score, that sensor receives high importance.

SHAP

SHAP is a contribution-based explainability method derived from game theory.

It estimates how much each feature contributes to the anomaly score.

SHAP perturbs or removes features and observes how prediction changes. Features causing large prediction changes receive high SHAP values.

LEMNA

LEMNA is a local surrogate explainability method.

It approximates the neural network locally around an anomaly point using a simpler interpretable model.

It:

- Generates neighboring perturbed samples
- Observes model outputs
- Builds a local approximation
- Extracts feature importance from the local model.

Evaluation

The TEP dataset already specifies which sensor was attacked.

Each explainability method produces sensor importance scores.

Sensors are ranked according to importance.

If the attacked sensor receives:

- Rank 1 → good attribution
- Higher rank → poorer attribution.

The paper reports average ranking across many attack samples.

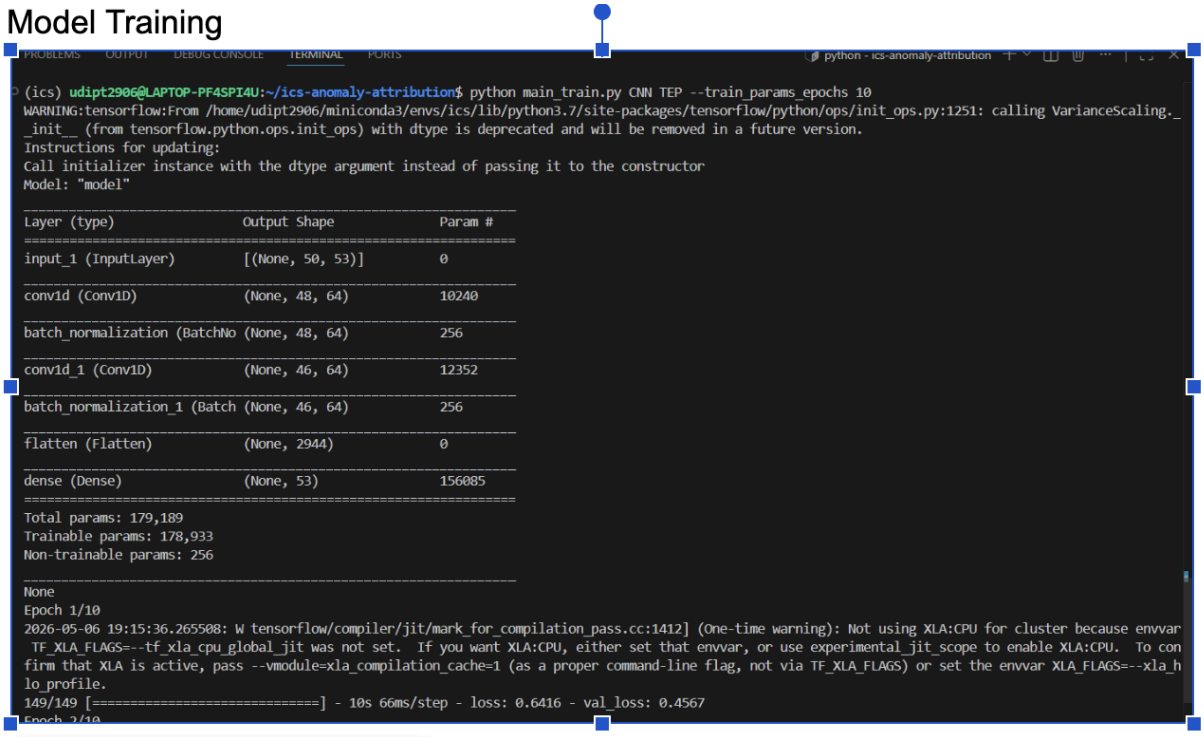
The original pipeline is:

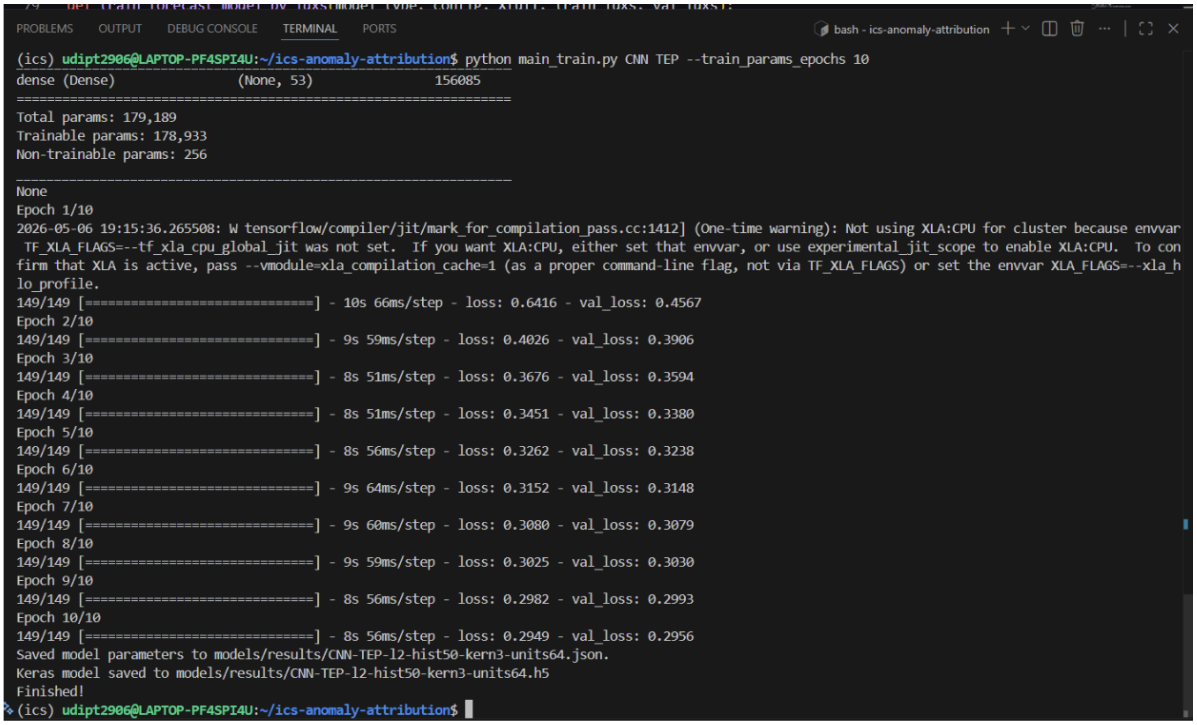
CNN
↓
Prediction Error
↓
External Explainability

This introduces:

- Additional computational overhead
- Separate post-hoc explainability stage
- Attribution instability.

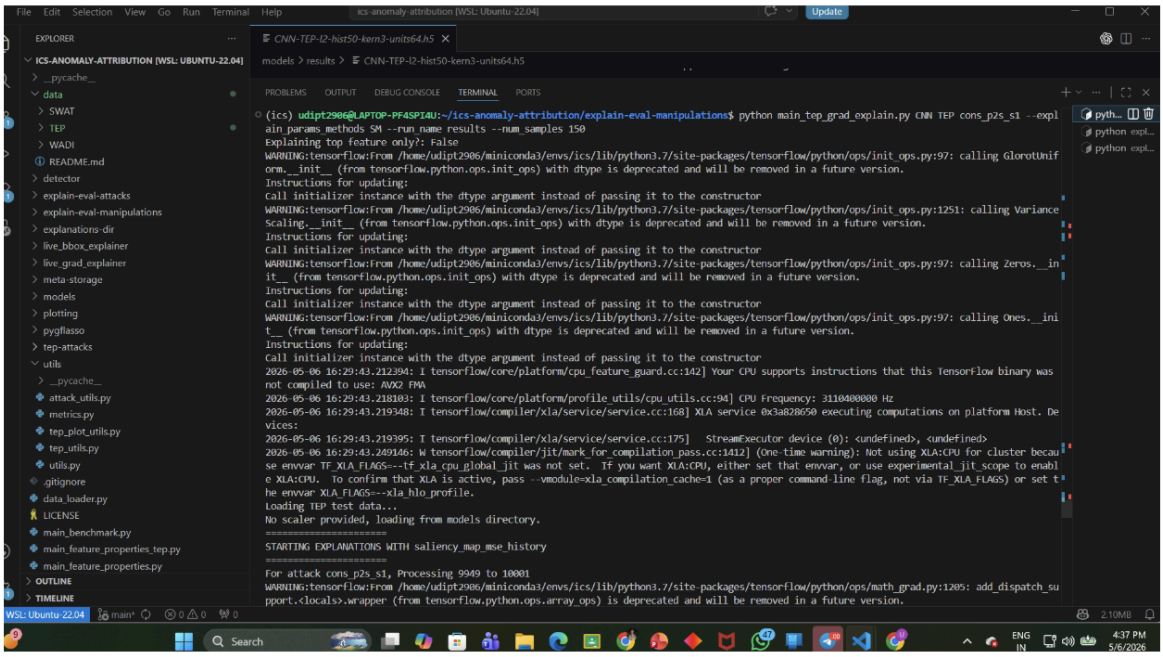
Model Training





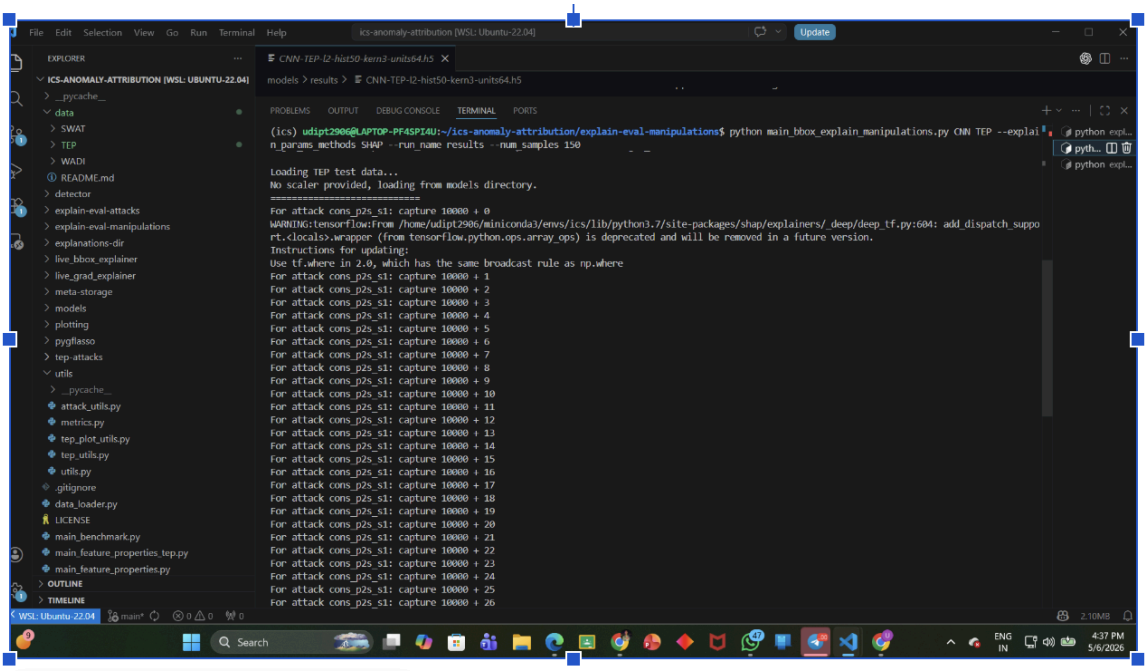
Error: 0.2956

Explanation generation

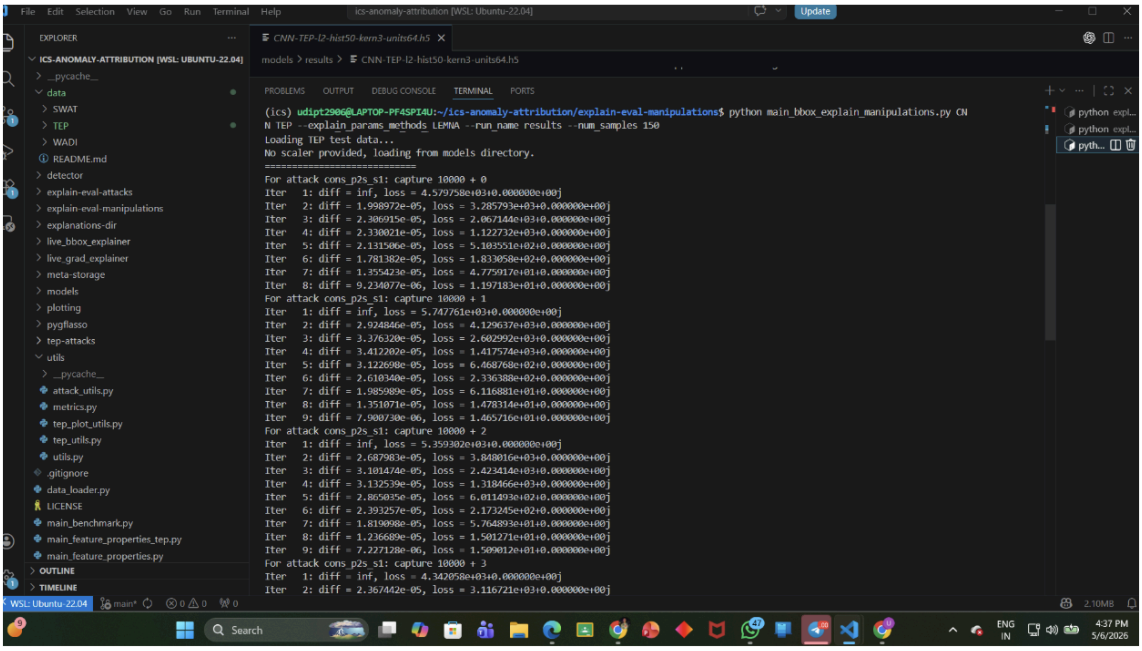


.

SM

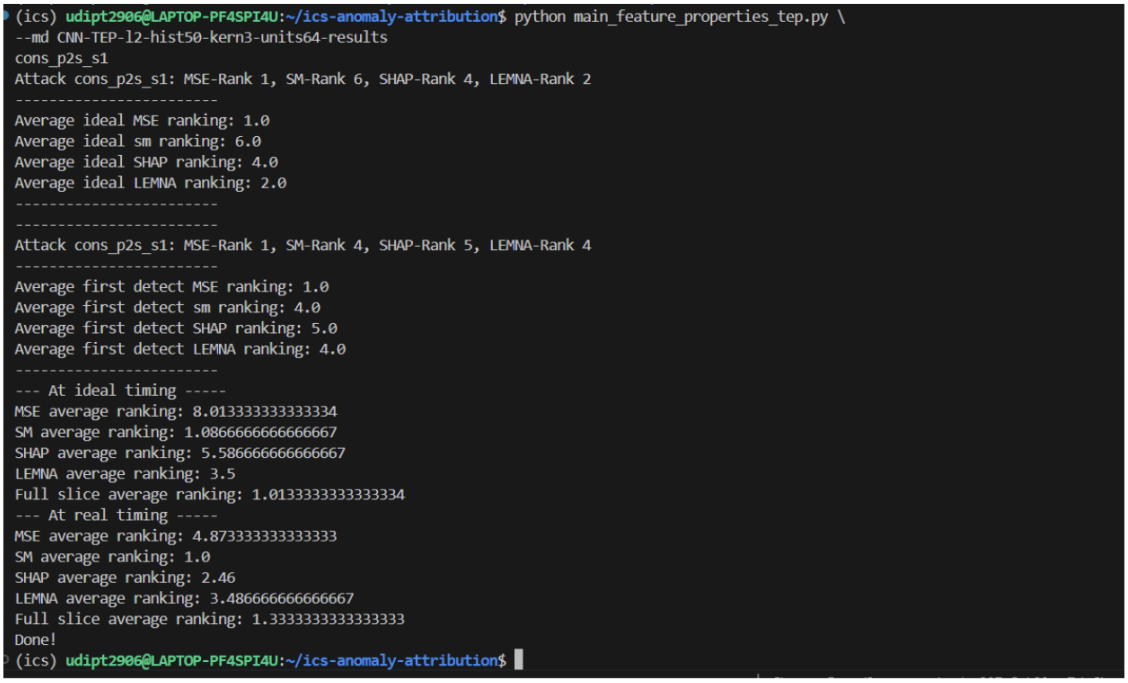

ics-anomaly-attribution [WSL: Ubuntu-22.04]
Update
CNN-TEP-l2-hist50-kern3-units64.h5
models > results > CNN-TEP-l2-hist50-kern3-units64.h5
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(ics) udipt2906@LAPTOP-PF4SPI4U:~/ics-anomaly-attribution/explain-eval-manipulations$ python main_bbox_explain_manipulations.py CNN TEP --explain_params_methods SHAP --run_name results --num_samples 150
Loading TEP test data...
No scaler provided, loading from models directory.
==========
For attack cons_p2s_s1: capture 10000 + 0
WARNING:tensorflow:From /home/udipt2906/miniconda3/envs/ics/lib/python3.7/site-packages/shap/explainers/_deep/deep_tf.py:604: add_dispatch_support.<locals>.wrapper (from tensorflow.python.ops.array_ops) is deprecated and will be removed in a future version.
Instructions for updating:
Use tf.where in 2.0, which has the same broadcast rule as np.where
For attack cons_p2s_s1: capture 10000 + 1
For attack cons_p2s_s1: capture 10000 + 2
For attack cons_p2s_s1: capture 10000 + 3
For attack cons_p2s_s1: capture 10000 + 4
For attack cons_p2s_s1: capture 10000 + 5
For attack cons_p2s_s1: capture 10000 + 6
For attack cons_p2s_s1: capture 10000 + 7
For attack cons_p2s_s1: capture 10000 + 8
For attack cons_p2s_s1: capture 10000 + 9
For attack cons_p2s_s1: capture 10000 + 10
For attack cons_p2s_s1: capture 10000 + 11
For attack cons_p2s_s1: capture 10000 + 12
For attack cons_p2s_s1: capture 10000 + 13
For attack cons_p2s_s1: capture 10000 + 14
For attack cons_p2s_s1: capture 10000 + 15
For attack cons_p2s_s1: capture 10000 + 16
For attack cons_p2s_s1: capture 10000 + 17
For attack cons_p2s_s1: capture 10000 + 18
For attack cons_p2s_s1: capture 10000 + 19
For attack cons_p2s_s1: capture 10000 + 20
For attack cons_p2s_s1: capture 10000 + 21
For attack cons_p2s_s1: capture 10000 + 22
For attack cons_p2s_s1: capture 10000 + 23
For attack cons_p2s_s1: capture 10000 + 24
For attack cons_p2s_s1: capture 10000 + 25
For attack cons_p2s_s1: capture 10000 + 26

SHAP


ics-anomaly-attribution [WSL: Ubuntu-22.04]
Update
CNN-TEP-l2-hist50-kern3-units64.h5
models > results > CNN-TEP-l2-hist50-kern3-units64.h5
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(ics) udipt2906@LAPTOP-PF4SPI4U:~/ics-anomaly-attribution/explain-eval-manipulations$ python main_bbox_explain_manipulations.py CNN TEP --explain_params_methods LEMNA --run_name results --num_samples 150
Loading TEP test data...
No scaler provided, loading from models directory.
==========
For attack cons_p2s_s1: capture 10000 + 0
Iter 1: diff = inf, loss = 4.579758e+03+0.000000e+00j
Iter 2: diff = 1.998972e-05, loss = 3.285793e+03+0.000000e+00j
Iter 3: diff = 2.306915e-05, loss = 2.067144e+03+0.000000e+00j
Iter 4: diff = 2.330021e-05, loss = 1.122732e+03+0.000000e+00j
Iter 5: diff = 2.131506e-05, loss = 5.103551e+02+0.000000e+00j
Iter 6: diff = 1.781382e-05, loss = 1.833058e+02+0.000000e+00j
Iter 7: diff = 1.355423e-05, loss = 4.775917e+01+0.000000e+00j
Iter 8: diff = 9.234077e-06, loss = 1.197183e+01+0.000000e+00j
For attack cons_p2s_s1: capture 10000 + 1
Iter 1: diff = inf, loss = 5.747761e+03+0.000000e+00j
Iter 2: diff = 2.924846e-05, loss = 4.129637e+03+0.000000e+00j
Iter 3: diff = 3.376320e-05, loss = 2.602992e+03+0.000000e+00j
Iter 4: diff = 3.412202e-05, loss = 1.417574e+03+0.000000e+00j
Iter 5: diff = 3.122698e-05, loss = 6.468768e+02+0.000000e+00j
Iter 6: diff = 2.610340e-05, loss = 2.336388e+02+0.000000e+00j
Iter 7: diff = 1.985089e-05, loss = 6.116881e+01+0.000000e+00j
Iter 8: diff = 1.351071e-05, loss = 1.478314e+01+0.000000e+00j
Iter 9: diff = 7.900730e-06, loss = 1.465716e+01+0.000000e+00j
For attack cons_p2s_s1: capture 10000 + 2
Iter 1: diff = inf, loss = 5.359302e+03+0.000000e+00j
Iter 2: diff = 2.687983e-05, loss = 3.848016e+03+0.000000e+00j
Iter 3: diff = 3.101474e-05, loss = 2.423414e+03+0.000000e+00j
Iter 4: diff = 3.132539e-05, loss = 1.318466e+03+0.000000e+00j
Iter 5: diff = 2.865035e-05, loss = 6.011493e+02+0.000000e+00j
Iter 6: diff = 2.393257e-05, loss = 2.173245e+02+0.000000e+00j
Iter 7: diff = 1.819098e-05, loss = 5.764893e+01+0.000000e+00j
Iter 8: diff = 1.236689e-05, loss = 1.501271e+01+0.000000e+00j
Iter 9: diff = 7.227128e-06, loss = 1.509012e+01+0.000000e+00j
For attack cons_p2s_s1: capture 10000 + 3
Iter 1: diff = inf, loss = 4.342058e+03+0.000000e+00j
Iter 2: diff = 2.367442e-05, loss = 3.116721e+03+0.000000e+00j

LEMNA

Final results of CNN model:



Proposed Novelty: CNN_ATTN

The proposed extension introduces feature-time attention into the CNN forecasting model.

Instead of relying completely on external explainability methods, the model internally learns important temporal-feature regions during prediction.

Architecture:

Input
↓
Conv1D Feature Extraction
↓
Feature-Time Attention
↓
Attention-Weighted Representation
↓
Temporal Pooling
↓
Forecast Output

Attention acts as a focus mechanism.

It learns which timesteps and features are more important during prediction.

To align attention with anomaly detection, gradient-based attribution is combined with attention.

Final attribution:

Attribution = |Gradient × Input| × Attention

Here:

- Gradient identifies which inputs strongly affect anomaly error
- Attention identifies where the model internally focuses.

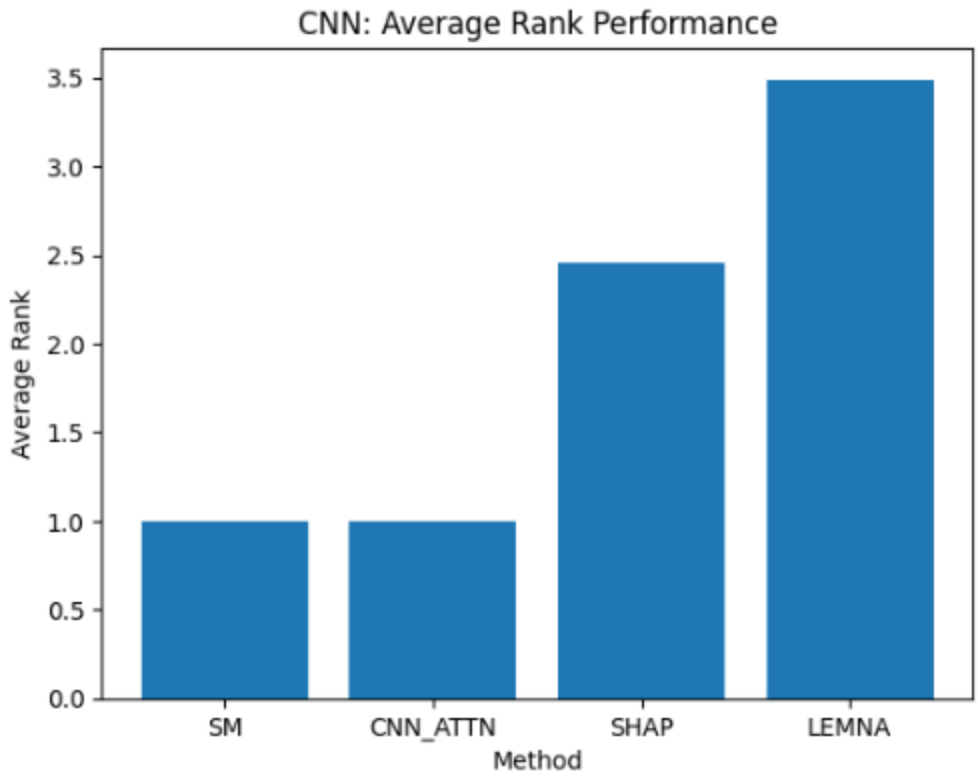
The final attribution highlights regions that are:

- Important for prediction
- Important for anomaly error.

The resulting method creates a more unified detection and attribution pipeline compared to fully external explainability methods.

Final ranking performance:

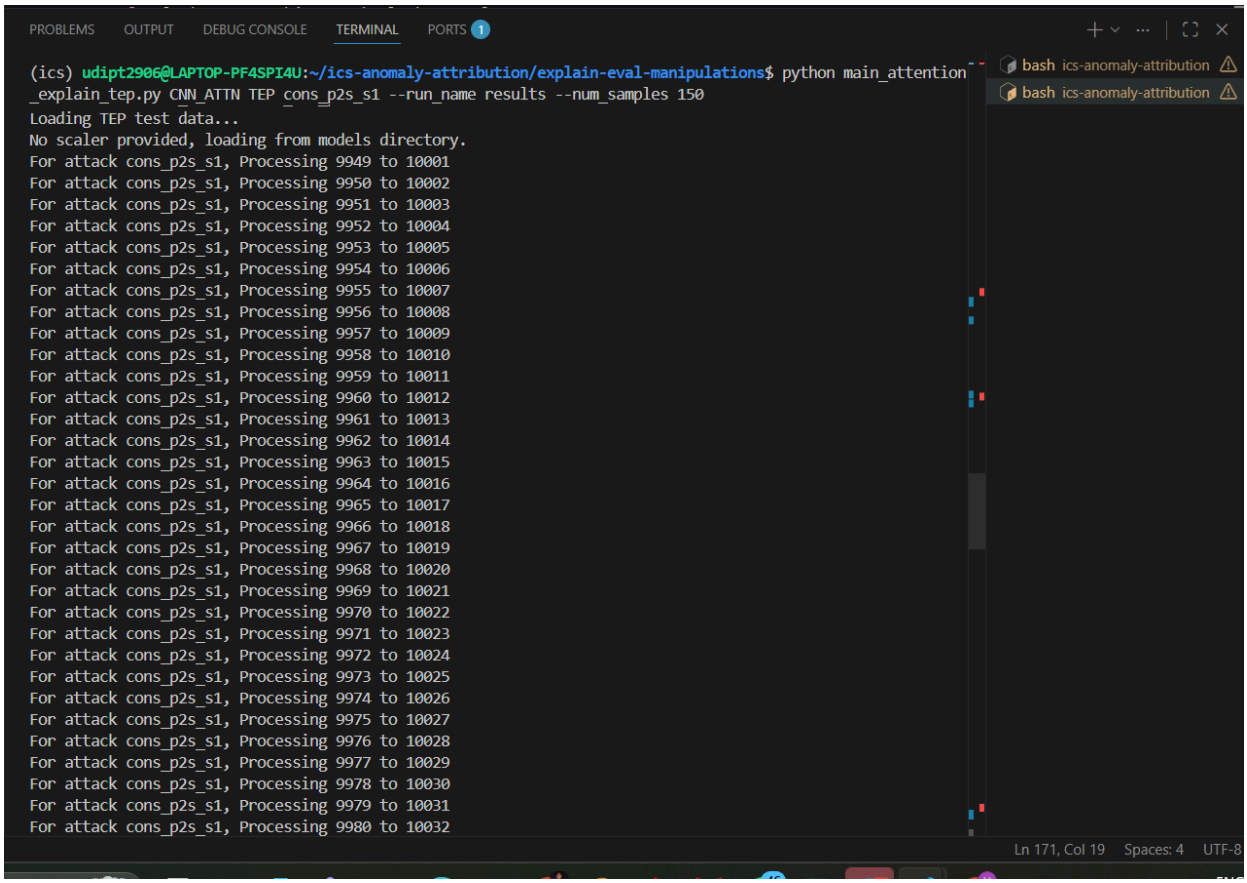
Method	Average Rank
SM	≈ 1
CNN_ATTN	≈ 1
SHAP	≈ 2.46
LEMNA	≈ 3.49

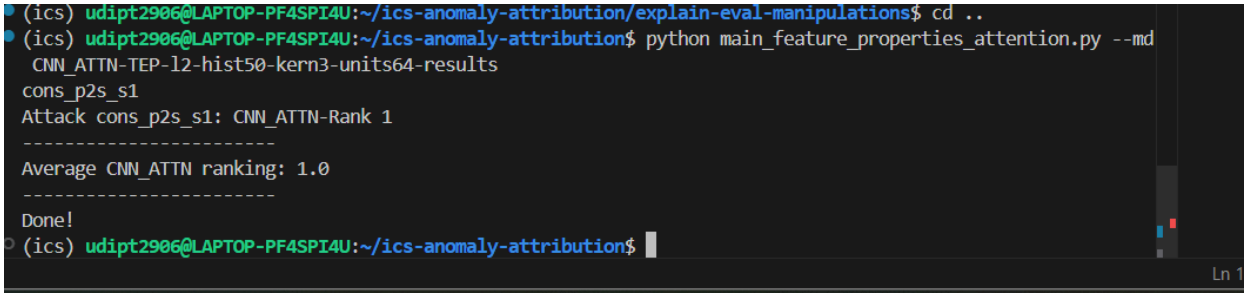


The results show that attention-guided attribution achieves performance comparable to the best post-hoc explainability baseline while integrating attribution more closely into the model architecture

Novelty:

```
(ics) udipt2906@LAPTOP-PF4SPI4U:~/ics-anomaly-attribution$ python main_train2.py CNN_ATTN TEP --train_param
s_epochs 10
Instructions for updating:
Call initializer instance with the dtype argument instead of passing it to the constructor
Epoch 1/10
2026-05-06 18:43:22.420187: W tensorflow/compiler/jit/mark_for_compilation_pass.cc:1412] (One-time warning)
: Not using XLA:CPU for cluster because envvar TF_XLA_FLAGS=--tf_xla_cpu_global_jit was not set.  If you wa
nt XLA:CPU, either set that envvar, or use experimental_jit_scope to enable XLA:CPU.  To confirm that XLA i
s active, pass --vmodule=xla_compilation_cache=1 (as a proper command-line flag, not via TF_XLA_FLAGS) or s
et the envvar XLA_FLAGS=--xla_hlo_profile.
149/149 [==============================] - 10s 66ms/step - loss: 0.5694 - val_loss: 0.4461
Epoch 2/10
149/149 [==============================] - 8s 54ms/step - loss: 0.3790 - val_loss: 0.3804
Epoch 3/10
149/149 [==============================] - 8s 56ms/step - loss: 0.3549 - val_loss: 0.3612
Epoch 4/10
149/149 [==============================] - 8s 54ms/step - loss: 0.3462 - val_loss: 0.3478
Epoch 5/10
149/149 [==============================] - 8s 54ms/step - loss: 0.3414 - val_loss: 0.3416
Epoch 6/10
149/149 [==============================] - 8s 53ms/step - loss: 0.3378 - val_loss: 0.3376
Epoch 7/10
149/149 [==============================] - 8s 55ms/step - loss: 0.3345 - val_loss: 0.3341
Epoch 8/10
149/149 [==============================] - 8s 55ms/step - loss: 0.3311 - val_loss: 0.3308
Epoch 9/10
149/149 [==============================] - 8s 53ms/step - loss: 0.3271 - val_loss: 0.3269
Epoch 10/10
149/149 [==============================] - 8s 54ms/step - loss: 0.3226 - val_loss: 0.3225
Saved model parameters to models/results/CNN_ATTN-TEP-l2-hist50-kern3-units64.json.
Keras model saved to models/results/CNN_ATTN-TEP-l2-hist50-kern3-units64.h5
Finished!
```


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS 1
bash ics-anomaly-attribution
bash ics-anomaly-attribution
(ics) udipt2906@LAPTOP-PF4SPI4U:~/ics-anomaly-attribution/explain-eval-manipulations$ python main_attention_explain_tep.py CNN_ATTN TEP cons_p2s_s1 --run_name results --num_samples 150
Loading TEP test data...
No scaler provided, loading from models directory.
For attack cons_p2s_s1, Processing 9949 to 10001
For attack cons_p2s_s1, Processing 9950 to 10002
For attack cons_p2s_s1, Processing 9951 to 10003
For attack cons_p2s_s1, Processing 9952 to 10004
For attack cons_p2s_s1, Processing 9953 to 10005
For attack cons_p2s_s1, Processing 9954 to 10006
For attack cons_p2s_s1, Processing 9955 to 10007
For attack cons_p2s_s1, Processing 9956 to 10008
For attack cons_p2s_s1, Processing 9957 to 10009
For attack cons_p2s_s1, Processing 9958 to 10010
For attack cons_p2s_s1, Processing 9959 to 10011
For attack cons_p2s_s1, Processing 9960 to 10012
For attack cons_p2s_s1, Processing 9961 to 10013
For attack cons_p2s_s1, Processing 9962 to 10014
For attack cons_p2s_s1, Processing 9963 to 10015
For attack cons_p2s_s1, Processing 9964 to 10016
For attack cons_p2s_s1, Processing 9965 to 10017
For attack cons_p2s_s1, Processing 9966 to 10018
For attack cons_p2s_s1, Processing 9967 to 10019
For attack cons_p2s_s1, Processing 9968 to 10020
For attack cons_p2s_s1, Processing 9969 to 10021
For attack cons_p2s_s1, Processing 9970 to 10022
For attack cons_p2s_s1, Processing 9971 to 10023
For attack cons_p2s_s1, Processing 9972 to 10024
For attack cons_p2s_s1, Processing 9973 to 10025
For attack cons_p2s_s1, Processing 9974 to 10026
For attack cons_p2s_s1, Processing 9975 to 10027
For attack cons_p2s_s1, Processing 9976 to 10028
For attack cons_p2s_s1, Processing 9977 to 10029
For attack cons_p2s_s1, Processing 9978 to 10030
For attack cons_p2s_s1, Processing 9979 to 10031
For attack cons_p2s_s1, Processing 9980 to 10032
Ln 171, Col 19
Spaces: 4
UTF-8


(ics) udipt2906@LAPTOP-PF4SPI4U:~/ics-anomaly-attribution/explain-eval-manipulations$ cd ..
(ics) udipt2906@LAPTOP-PF4SPI4U:~/ics-anomaly-attribution$ python main_feature_properties_attention.py --md CNN_ATTN-TEP-l2-hist50-kern3-units64-results
cons_p2s_s1
Attack cons_p2s_s1: CNN_ATTN-Rank 1
Average CNN_ATTN ranking: 1.0
Done!
(ics) udipt2906@LAPTOP-PF4SPI4U:~/ics-anomaly-attribution$